

CSCE 641 COMPUTER GRAPHICS · MODULE 09 · LECTURE NOTES

Sampling, Aliasing, and Antialiasing

The theory behind every artifact in this course.

MODULE OUTCOMES

- State the sampling theorem and apply it to rendering.
- Explain aliasing as frequency folding, not as 'jaggies'.
- Compare supersampling, MSAA, and post-process AA honestly.
- Explain why MSAA is cheap and why deferred rendering breaks it.
- Choose an antialiasing strategy for a given renderer and justify it.

1 The unifying diagnosis

Module 01 claimed that nearly every artifact in this course is a sampling failure. This module makes that precise, and the payoff is that five apparently unrelated bugs become one phenomenon with one body of theory behind it.

Symptom	Signal	Sampling rate
Jagged, stair-stepped edges	Coverage: a step function at the triangle boundary	One sample per pixel, at the centre.
Textures shimmer and crawl in motion	The texture image projected onto the surface	One sample per pixel; the footprint may span hundreds of texels.
Specular highlights vanish and pop	A narrow $\cos^n$ lobe over the surface	Three samples per triangle, under Gouraud shading.
Thin geometry flickers and breaks up	Coverage of a feature smaller than a pixel	One sample per pixel, which the feature may fall between.
Shadow edges are blocky and stair-stepped	The depth function of the scene from the light	One sample per shadow-map texel.

2 What aliasing actually is

The word is often used loosely to mean 'jagged edges'. The real definition is more useful because it predicts the artifacts you have not seen yet.

THE DEFINITION

When a signal is sampled at rate f_s , any frequency component above $f_s/2$ does not disappear. It reappears — is *aliased* — as a lower frequency, specifically at $|f - n \cdot f_s|$.

This is why aliasing artifacts are structured rather than noisy: a moiré pattern on a distant fence is the fence's high-frequency detail folded down into a coarse pattern that does not exist in the scene.

It is also why aliasing moves coherently and distractingly with the camera: as the sampling grid shifts relative to the signal, the folded frequencies shift too.

2.1 Nyquist, and why rendering violates it

$$f_s > 2 f_{\max}$$

Sample at more than twice the highest frequency present and the signal can be reconstructed exactly. Sample below it and information is irrecoverably corrupted — not merely lost, but replaced by false low-frequency content.

Rendering has a structural problem with this. A triangle edge is a step function: the coverage goes from 0 to 1 discontinuously. A step function has energy at *all* frequencies, without bound. There is no sampling rate high enough. Doubling the resolution halves the visible stair step and does not remove it.

So antialiasing in rendering is not 'satisfy Nyquist'. It is 'suppress the artifact to below the threshold of notice, as cheaply as possible'. Knowing that reframes every technique below as damage control, which is what they are.

3 The two strategies**3.1 Prefiltering**

Remove the high frequencies before sampling. If the signal is band-limited below $f_s/2$ then one sample per pixel is correct and nothing folds. This is ideal, and it is available exactly when you know the signal in advance.

Mipmapping is prefiltering. Each mip level is a band-limited version of the texture, computed offline, and level selection chooses the band limit appropriate to the current sampling rate. This is why mipmapping works so well and costs so little: the expensive part happened before the frame started.

It is not available for geometric coverage, because coverage depends on the camera and cannot be precomputed. That asymmetry explains why texture aliasing is a solved problem and edge aliasing is not.

3.2 Supersampling

Raise the sampling rate, then average down. This works on any signal, known in advance or not. The cost scales directly with the sample count, and because step edges have unbounded bandwidth it never eliminates aliasing — it pushes the folding up to a frequency where it is less visible.

4 The methods

4.1 SSAA

Render the whole frame at $N\times$ resolution and downsample. Antialiases everything — coverage, shading, textures, specular highlights, alpha test — because every part of the pipeline genuinely runs at the higher rate. It is also $N\times$ the cost in fragments, bandwidth, and memory. $4\times$ SSAA is approximately a quarter of your frame rate.

Almost never used in real time, but it is the reference you compare everything else against, and you should implement it once so you know what correct looks like.

4.2 MSAA

THE INSIGHT THAT MAKES MSAA CHEAP

Geometric edges alias because *coverage* is undersampled. The shading *inside* a triangle is usually smooth and does not need a higher rate.

So MSAA stores N depth and coverage samples per pixel, but invokes the fragment shader only **once per pixel per triangle**, writing that single result to every sample the triangle covers.

Coverage resolution goes up $N\times$; shading cost stays at $1\times$. Only memory and bandwidth scale. For a forward renderer with smooth shading this is close to a free lunch, and it is why MSAA dominated for fifteen years.

Its limitations follow directly from the same insight:

- **It does not touch shader aliasing.** A high-frequency normal map producing sparkling specular highlights is sampled once per pixel regardless of MSAA level, because the shader ran once.
- **Alpha-tested edges are invisible to it.** The visual edge of a leaf card is created by discard inside the shader, not by triangle coverage. *Alpha-to-coverage* partially recovers this by converting alpha into a coverage mask.
- **Deferred rendering breaks it.** In a deferred renderer lighting happens in a later full-screen pass, by which point the G-buffer has been resolved to one value per pixel. Keeping per-sample G-buffers costs $N\times$ the memory and bandwidth of the G-buffer, which defeats the reason for deferring in the first place.

4.3 Post-process AA (FXAA, SMAA)

Analyse the finished image, find edge-like patterns, and blur across them. Extremely cheap — around a millisecond — and independent of how the frame was rendered, so it works with deferred shading, alpha test, and anything else.

The fundamental limitation is that it has no information that was not in the final image. It cannot recover a sub-pixel feature that was never sampled, and it cannot distinguish a genuine edge from a deliberate high-contrast texture detail, so it blurs some things that should stay sharp. It is a plausible-looking

smoothing, not a reconstruction.

4.4 TAA

Temporal antialiasing jitters the projection matrix by a sub-pixel offset each frame, so successive frames sample the scene at different positions. Previous frames are reprojected into the current frame using motion vectors and blended in. Over several frames this accumulates many samples per pixel for the cost of one sample per frame — supersampling spread across time.

Crucially, it antialiases *shading* as well as geometry, because the whole pipeline runs at each jittered position. That is why TAA displaced MSAA as modern rendering became more shading-limited and more deferred.

WHAT TAA COSTS

Ghosting. When reprojection is wrong — a shadow moves without the object, a reflection changes, a transparent surface has no sensible motion vector — stale colour trails behind.

Blur in motion. The history buffer is resampled every frame, and repeated bilinear resampling is a low-pass filter. Movement progressively softens the image.

Disocclusion. Pixels revealed from behind an object have no history at all and must fall back to a single sample, so newly exposed regions alias visibly.

Neighbourhood clamping, variance clipping, and sharpening mitigate all three without removing them. DLSS and FSR are this same architecture with learned or hand-tuned reconstruction, and they inherit the same failure modes in better-disguised form.

5 Choosing

Situation	Choice	Why
Forward renderer, smooth shading	MSAA 4×	Near-free for what it fixes, and no temporal artifacts at all.
Deferred renderer	TAA	MSAA is impractical; TAA also handles the shading aliasing that deferred pipelines tend to have.
Heavy specular, dense normal maps	TAA + specular antialiasing	The aliasing is in the shading, so MSAA cannot reach it. Clamp roughness and use normal-map mip bias as well.
Mobile or very tight budget	FXAA or SMAA	About a millisecond, no history buffer, no motion vectors.
Offline or reference image	SSAA	Correct, simple, and you have the time. Use it to judge everything else.

THE BEST ANTIALIASING HAPPENS BEFORE SAMPLING

Every method above is mitigation after the fact. Reducing the signal's bandwidth is better and often cheaper:

Clamp roughness to a minimum so specular lobes are never narrower than a pixel. Bias normal-map mip selection so distant surfaces lose high-frequency normals. Filter normal maps into roughness (LEAN or Toksvig mapping) so that shrinking geometry becomes rougher rather than sparklier.

These cost almost nothing and attack the cause rather than the symptom.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
GAMES101 Lectures 06 and 09 — Antialiasing, Texture aliasing	Aliasing presented properly in the frequency domain, with the spatial/frequency duality made visual.
CMU 15-462 — Sampling and Aliasing	The most rigorous free treatment: Fourier analysis applied directly to the rendering case.
Jorge Jimenez et al. — SMAA and 'Filmic SMAA' (SIGGRAPH course, free)	Post-process antialiasing done well, with an honest account of its limits.
Brian Karis — 'High Quality Temporal Supersampling' (SIGGRAPH course, free)	The reference presentation on practical TAA, including neighbourhood clamping. Still the clearest explanation of why TAA ghosts.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement $4\times$ and $16\times$ SSAA in your software rasterizer. Capture a scene with near-horizontal edges at $1\times$, $4\times$, and $16\times$ and measure the render time of each.
- 2 Implement MSAA in the software rasterizer: multiple coverage and depth samples per pixel, one shading evaluation. Verify that shading cost does not scale with sample count by counting shader invocations.
- 3 Construct a case where MSAA fails: a surface with a high-frequency normal map producing specular sparkle. Capture it at MSAA $1\times$ and $8\times$. Explain why they look the same.
- 4 Render a converging line pattern (a Siemens star or a fine picket fence receding to the horizon) and capture the moiré pattern. Then mipmap it properly and capture again.
- 5 Implement specular antialiasing by clamping roughness based on the screen-space derivative of the normal. Capture before and after on a sharp metallic surface in motion.

- 6 Write a one-page comparison of MSAA and TAA for a renderer you would actually build, with a recommendation and a justification.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define aliasing without using the word 'jagged'. Why do aliasing artifacts form patterns rather than noise?
- 2 State the Nyquist criterion and explain why geometric edges make it unsatisfiable in principle.
- 3 What are the two general strategies against aliasing? Give an example of each from this course.
- 4 Why is MSAA much cheaper than SSAA for the same edge quality, and what exactly does it fail to fix?
- 5 Give the specific reason deferred rendering makes MSAA impractical.
- 6 Name TAA's three characteristic artifacts and the underlying cause of each.
- 7 Give two ways to reduce aliasing by changing the signal rather than the sampling.